Final Exam, Faculty of Engineering, Chulalongkorn University
Course ID: 2110215 Course Name: Programming Methodology I
Second Semester, Date: 11 May 2020 Time: 13.00-16.00 (Online Exam)

Instructions

1. Your projects must be submitted as an assignment on MyCourseVille (please see "How to Submit").
2. Documents and files are allowed. Internet search is allowed (^_^).
3. No communication with another person is allowed.
4. Student must submit files before the time expires.
5. Any student who does not obey the regulations listed below will receive punishment under the Faculty of Engineering Official Announcement on January 6, 2003 regarding the exam regulations.
    a. With implicit evidence or showing intention for cheating, student will receive an F in that subject and will receive an academic suspension for 1 semester. (**หมายความว่า แค่ "สงสัย จากตัวโค้ดที่ทำ" ว่าลอกกัน อาจารย์มีสิทธิให้ F และพักการเรียนได้เลย**)
    b. With explicit evidence for cheating, student will receive an F in that subject and will receive an academic suspension for 1 year. (**หมายความว่า ถ้าเห็นว่ากำลังลอกเพื่อนอยู่ตอนนั้น ก็ F และพักการเรียน 1 ปี**)

Important Rules

- It is a student's responsibility to check the file. If it is corrupted or cannot be open, there is no score.

** Noted that Access Modifier Notations can be listed below*

+ (public)

(protected)

- (private)

<u>Underline</u> (static)

Italic (abstract)

Set-Up Instruction

- Set workspace to your usual workspace.
- All your exam files for each question must be in the project folder for that question.

How to Submit

There are 3 questions (hence 3 projects) in this exam.

- In your workspace directory, zip all 3 projects' folders into a zip file called yourID.zip (for example, 6233001121.zip).
- Submit the zip file as an assignment on MyCourseville.
- Make sure you do not include other files apart from files for each question. **A project that is too big in size may result in your submission NOT getting saved**.

Scoring (Total 30 points, will be scaled to 25 points)

- Part1 = 10 points
- Part2 = 9 points
- Part3 = 11 points

Part 2: JavaFX GUI

1. Objective

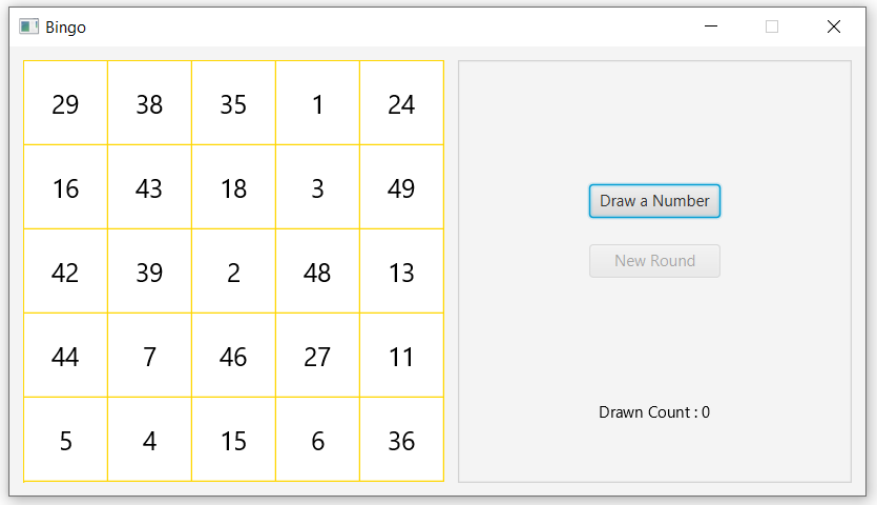
1) Students are able to implement GUI using JavaFx.

2. Instruction

1) Create Java Project named **"2110215_2_Final_Part2"**.
2) Copy all folders in **"toStudent/Part2"** to your project directory src folder.
3) You are to implement the following classes (detail for each class is given in section 3 and 4)

   a) **NumberSquare** (package gui)
   b) **ControlPane** (package gui)

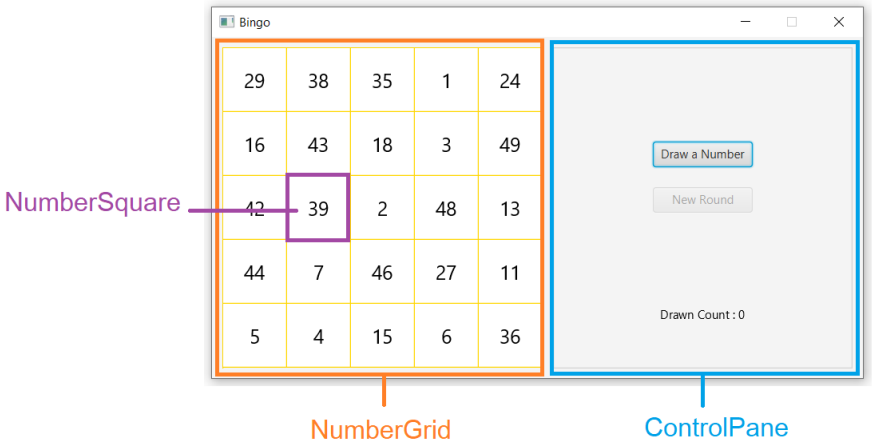
3. Problem Statement: Bingo



The initial GUI of the application.

You have seen a bingo game with alluring prize. With that alluring prize in mind, you develop a Bingo game to play and count how many times the number have to be drawn for you to bingo in a round. For analysis purpose, you have specified a random seed to random number generator to lock the bingo game results to be the same for each new application run. Possible numbers are 0-50.

4. Implementation Detail



Detailed GUI of the Application

The class package is summarized below.

*** In the following class description, only details of IMPORTANT fields and methods are given. ***

4.1 Package gui

4.1.1. public class **NumberSquare**: This class represents a number square.

Field

Name	Description
- int number	The number to display of this square.
- boolean isDrawn	This indicate whether the number of this square is already drawn.
- Text numberText	The Text object to display the number.

Constructor

Name	Description
+ NumberSquare ()	Constructor method. **This method is already provided.**

	Initializes with the following specifications: - Sets the alignment to **CENTER**. - Sets minimum and maximum width and height to 64. - Sets border to **GOLD** color stroke style **SOLID**, corner radii **EMPTY**, right and bottom widths to 1 and top and left widths to 0. - Initializes *numberText*, and set font with size 20.

Method

Name	**Description**
+ void setupNumber (int number)	/* FILL CODE */ This method is called when the square is assigned a new number. Does the following: - Sets the *number* field to the number parameter. - Sets *isDrawn* to false. - Sets text of *numberText* to string of the number parameter. - Sets the **Background** to be **filled** with **WHITE** color. **Hint:** class **Background** and **BackgroundFill** will be useful.
+ void highlight ()	/* FILL CODE */ Sets the **Background** to be **filled** with **ORANGE** color and sets *isDrawn* to true.
+ getter/setter for each field.	//only necessary ones are already provided.

4.1.2. public class **NumberGrid**: This class represents the number grid with NumberSquares.

This class is already provided.

Field

Name	Description
- ObservableList<NumberSquare> numberSquares	List that contains NumberSquare objects in the grid.

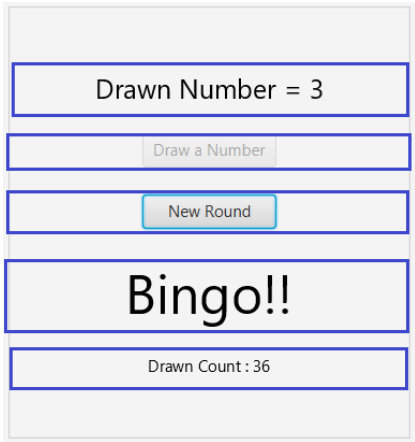
Constructor

Name	Description
+ NumberGrid ()	Constructor method. Initializes with the following specifications: - Sets border to **GOLD** color, stroke style **SOLID**, corner radii **EMPTY**, right and bottom widths to 0 and top and left widths to 1. - Initializes *numberSquares* by using **BingoUtil**.

Method

Name	Description
+ boolean highlightNumber (int drawnNumber)	This method highlights the number square that has the same number as the drawn number. Returns true if the grid is bingo after highlighting a number.
+ ObservableList<NumberSquare> getNumberSquares ()	Getter method for *numberSquares*.

4.1.3. public class **ControlPane**: This class is the pane that contains buttons and information texts. Items in the pane is arranged vertically.



Field

Name	Description
- Text drawnNumberText	The Text for displaying the number drawn.
- Text bingoText	The Text for displaying bingo text when the grid is bingo.
- Text drawCountText	The Text for displaying number drawn count.
- Button drawButton	The button for drawing a number.
- Button newRoundButton	The button for beginning a new round.
- NumberGrid numberGrid	A numberGrid that is updated by this ControlPane.

Constructor

Name	Description
+ ControlPane (NumberGrid numberGrid)	/* FILL CODE */ Constructor method. Sets *numberGrid* field to match the parameter. Then, initializes with the following specifications: - Sets the alignment to **CENTER**.

	- Sets preferred width to 300. - Sets spacing to 20. - Sets border to a border with border stroke **LIGHTGRAY** color, stroke style **SOLID**, corner radii **EMPTY**, border widths **DEFAULT**. - Initializes *drawnNumberText* and set its font with size 20. - Initializes *drawCountText*. - Initializes *bingoText* with initializeBingoText(). (See below) - Initializes *drawButton* with initializeDrawButton(). (See below) - Initializes *newRoundButton* with initializeNewRoundButton(). (See below) - Adds all Text and Button fields to this pane's children in correct order. - Sets *drawnNumberText*'s text and *drawCountText*'s text to beginning of a round texts by using **BingoUtil**.

Method

Name	Description
- void initializeBingoText()	/* FILL CODE */ - Initializes *bingoText* with text "Bingo!!" - Sets font with size 40 and set visible to false.
- void initializeDrawButton()	/* FILL CODE */ - Initializes *drawButton* with text "Draw a number". - Sets the button preferred width to 100. - Sets onAction to handle with drawButtonHandler() method. (See below)

- void initializeNewRoundButton()	/* FILL CODE */ - Initializes *newRoundButton* with text "New Round". - Sets the button preferred width to 100. - Sets disable to true. - Sets onAction to handle with newRoundButtonHandler() method. (See below)
- void drawButtonHandler ()	/* FILL CODE */ This method is the handler method for *drawButton*. Does the following: - Draws a random int number by using **BingoUtil** and highlights the drawn number in numberGrid. - If the numberGrid is bingo, set *bingoText* to visible, disable *drawButton* and enable *newRoundButton*. - Updates *drawnNumberText*'s text and *drawCountText*'s text by using **BingoUtil.**
- void newRoundButtonHandler ()	/* FILL CODE */ This method is the handler method for *newRoundButton*. Does the following: - Assigns random numbers to number squares by using **BingoUtil** - Sets *bingoText* to invisible, enable *drawButton* and disable *newRoundButton*. - Sets *drawnNumberText*'s text and *drawCountText*'s text to beginning of new round texts by using **BingoUtil**.
+ getter for each field.	/* FILL CODE */

4.1.4. public class **BingoUtil**: **This class is already provided.** Only useful public methods are shown here.

Method

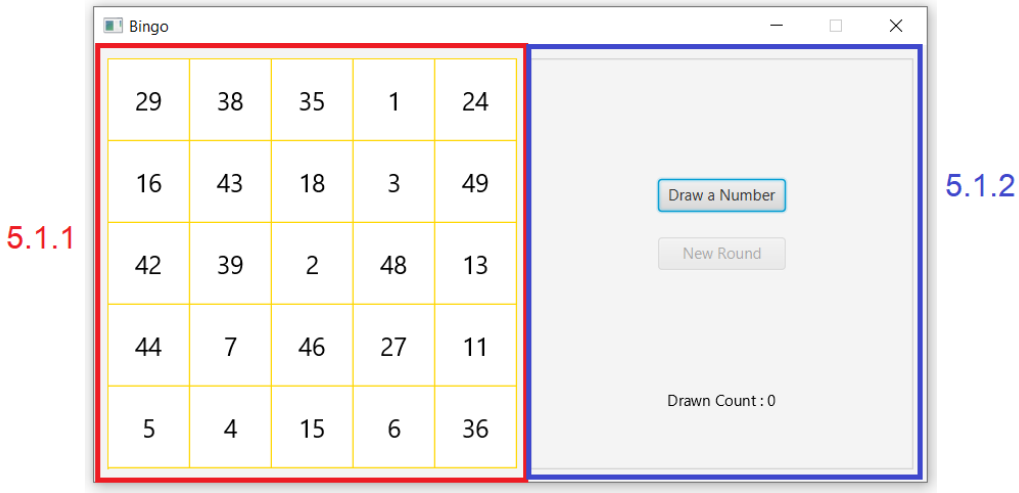
Name	**Description**
+ static void assignRandomNumbers (ObservableList<NumberSquare> numberSquares)	This method assigns random numbers to numberSquares passed in the parameter.
+ static void drawNumber ()	This method returns a random number that has not already drawn in the round.
+ static void updateTextsAfterDrawn (int drawnNumber, Text drawnNumberText, Text drawCountText)	This method update drawnNumberText's text according to drawnNumber and update drawCountText's text according to current drawn count
+ static void setTextsBeginning (Text drawnNumberText, Text drawCountText)	This method set drawNumberText's text and drawCountText's text at the beginning of a round.

4.2 Package main

4.2.1. public class **Main**: This class contains main method and is entry point of the application. **This class is already provided.**

5. Scoring Criteria (9 points)

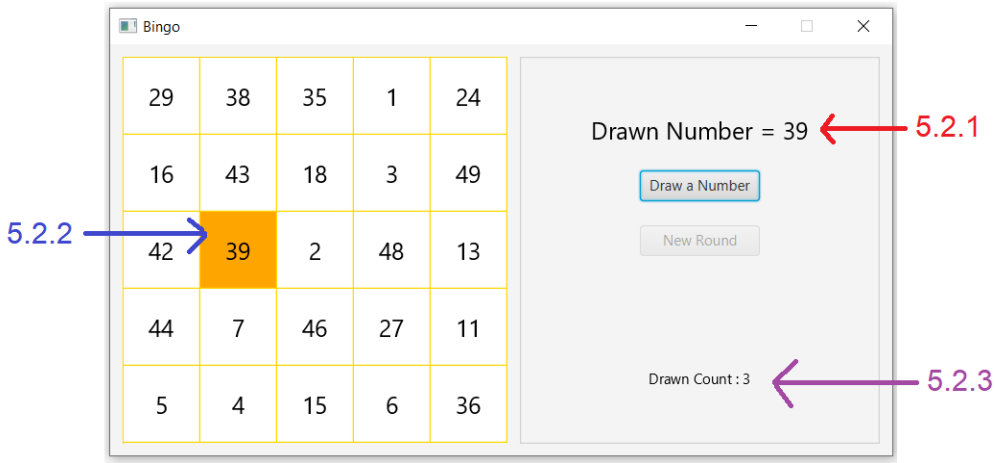
5.1 Initial



5.1.1. Initial numbers in squares grid are displayed properly. (1 point)

5.1.2. Initial buttons and texts are displayed properly. (1 point)

5.2 Draw

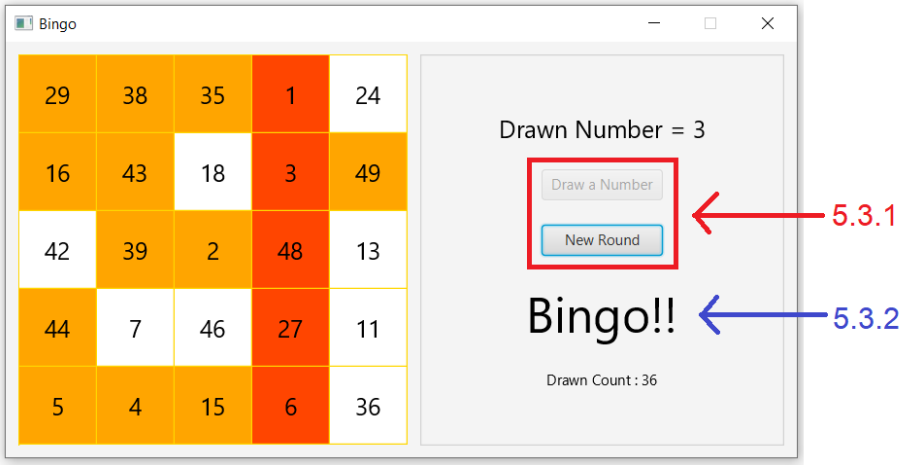


5.2.1 The drawn number text is displayed properly. (1 point)

5.2.2 The number square is highlighted properly when its number is drawn. (1 point)

5.2.3 The drawn count text is displayed properly. (1 point)

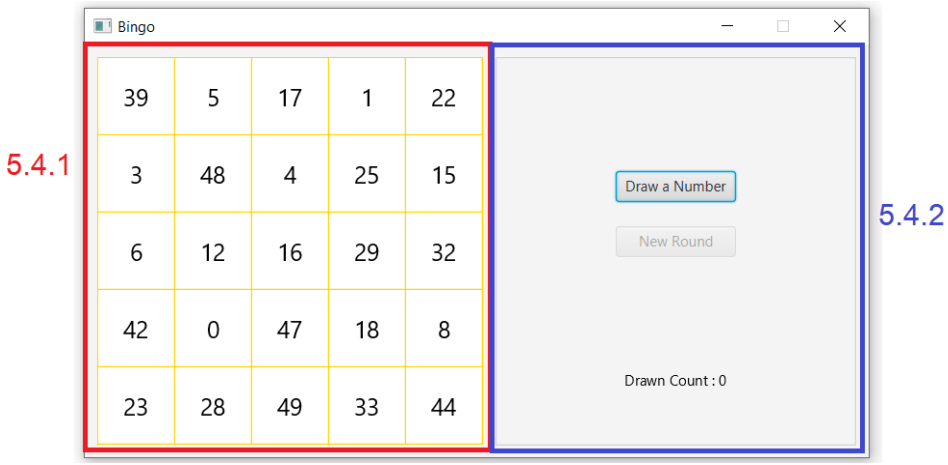
5.3 Bingo



5.3.1 Draw button is disabled and new round button is enabled when bingo. (1 point)

5.3.2 Bingo text is displayed when bingo. (1 point)

5.4 New round



5.4.1 New round button assigns new random numbers to squares. (1 point)

5.4.2 New round button enables draw button, disables itself, hides and shows text properly. (1 point)

Part 3: Java Thread

1. Objective

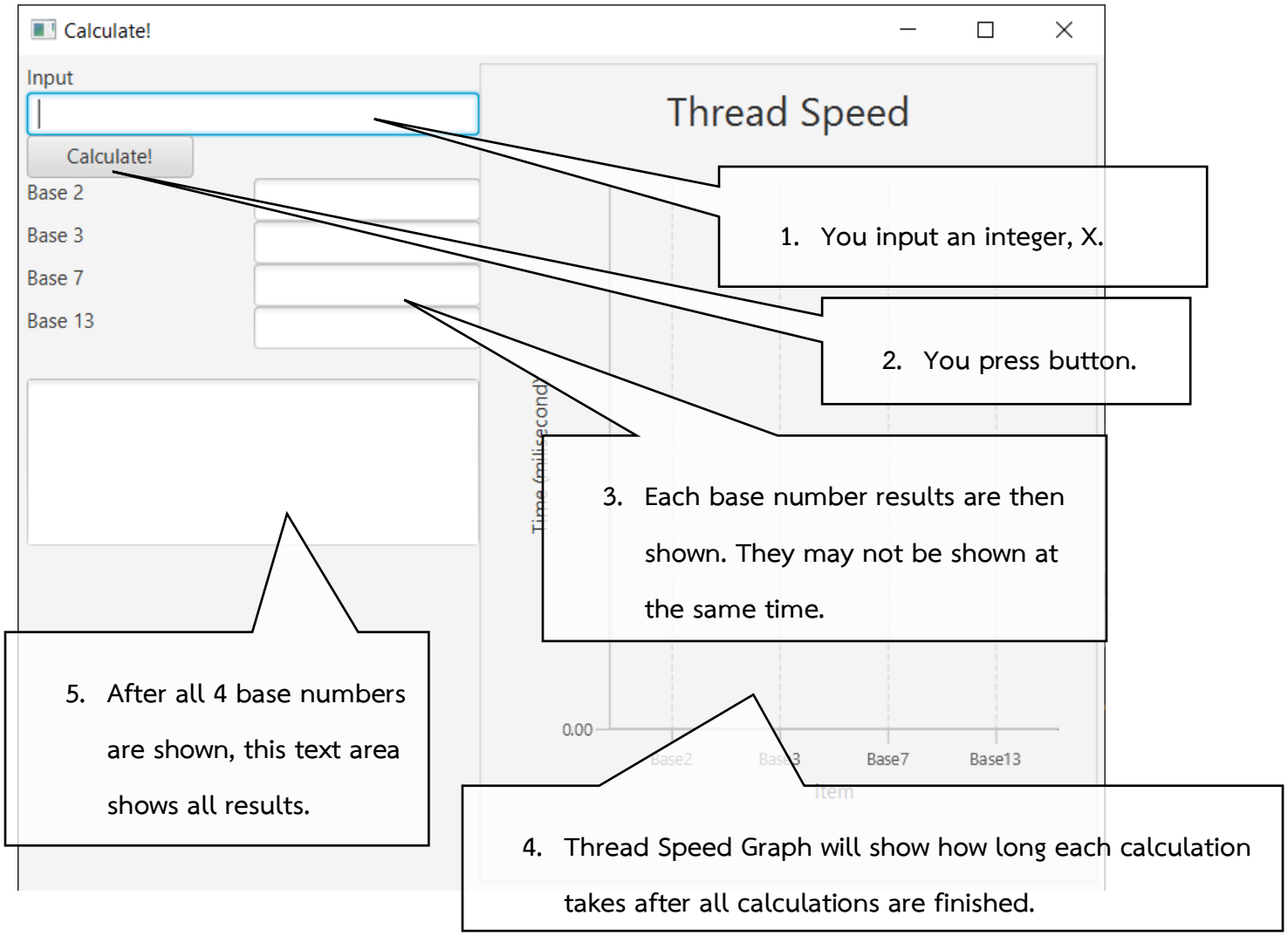
1) Be able to implement Java thread to make a program responsive.

2. Instruction

1) Create a new Java Project named "**2110215_2_Final_Part3**".
2) Copy folder "**BarChart**" and "**main**" (in "toStudent/Part3") into your project src folder.
3) Fix the code (detail shown below) inside.

3. Problem Statement: Calculating various base numbers.

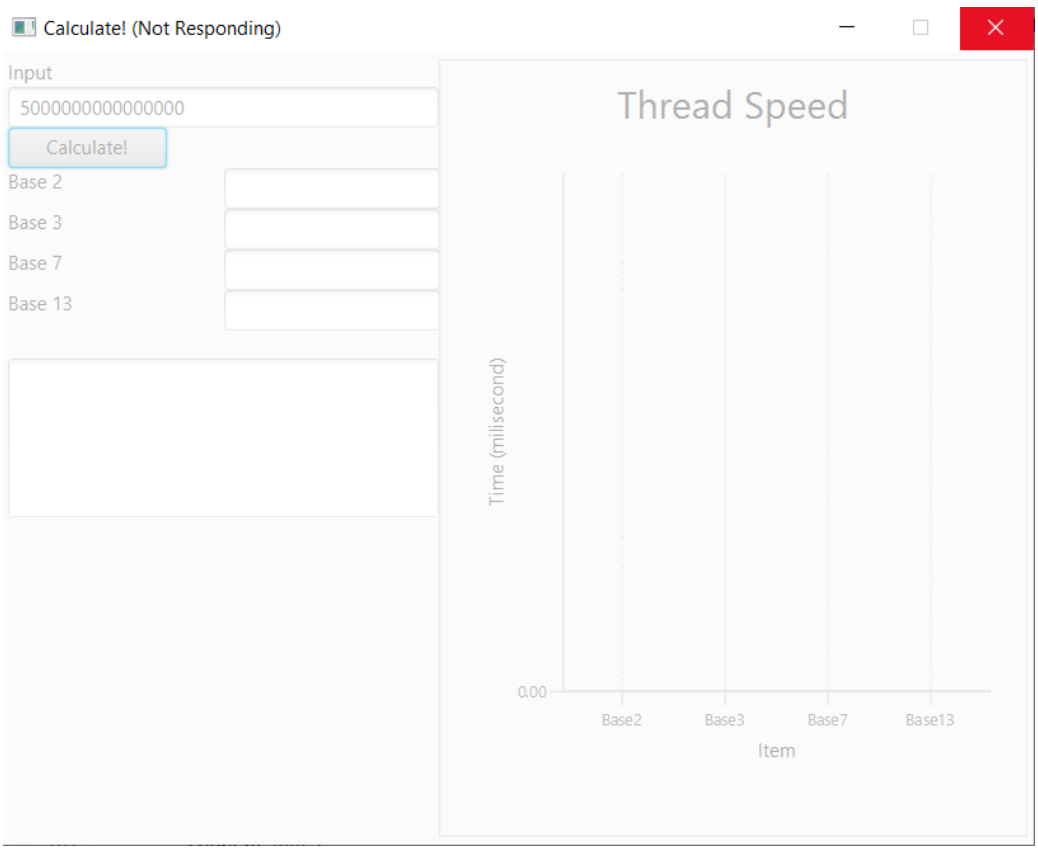
main.java is an application that calculates the value of different base number (Using **base_X(double x,int base)** method) including base 2, 3, 7, 13. The application works as follows:



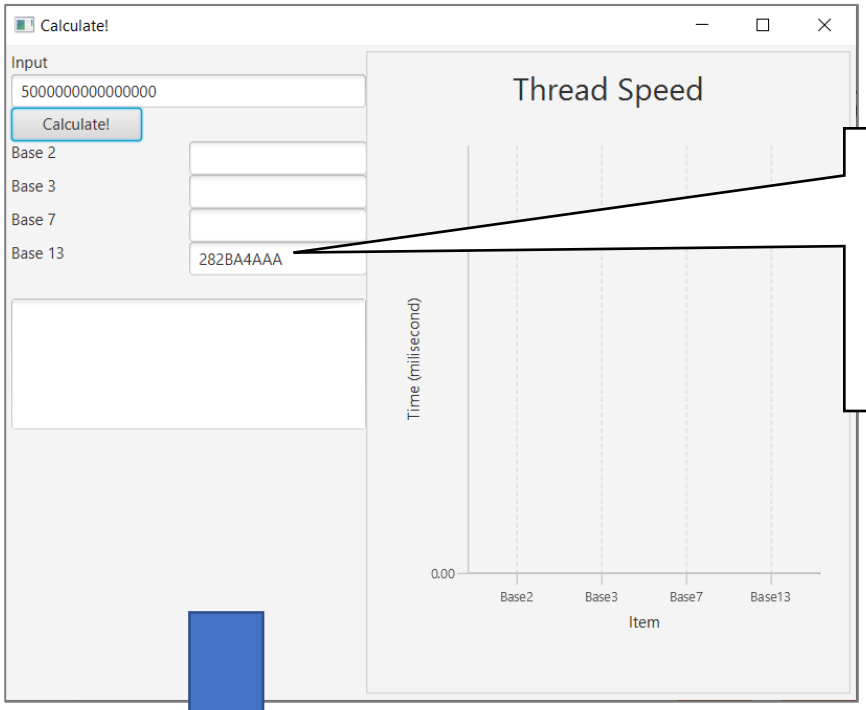
(Chart will show approximately how long each base number calculation should take individually. You don't have to change any code related to chart or run time. Chart should work the same way in a finished code as in the initial code)

Try out main.java with small input such as 50 or 100. You will see each result shown quickly one after the other.

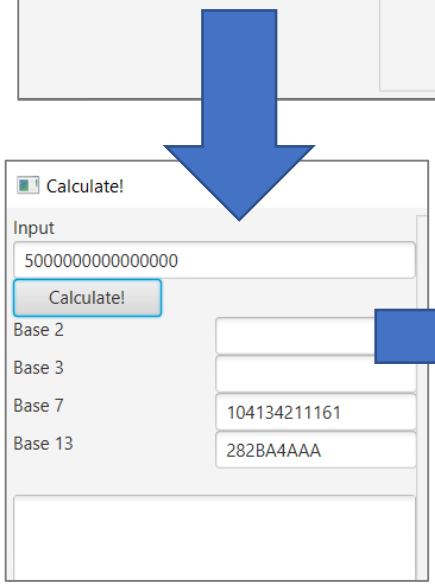
The problem is, when we try a larger number, such as 5000000000000000 (five thousand billon, 5×10^{15}), the user interface will stop responding for a while (as shown in the picture below). The Calculate button won't bounce back.

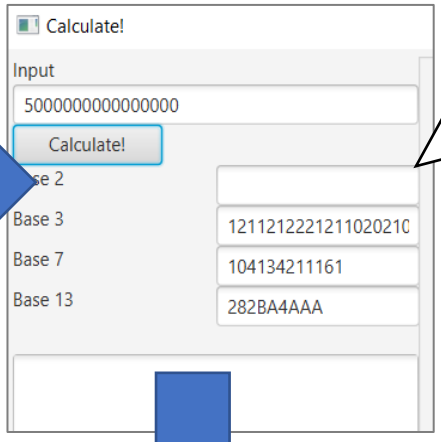


Your task is to use threads to allow the user interface to respond when Calculate button is clicked. After your fix, pressing Calculate when your input is 5000000000000000 will result in the following:

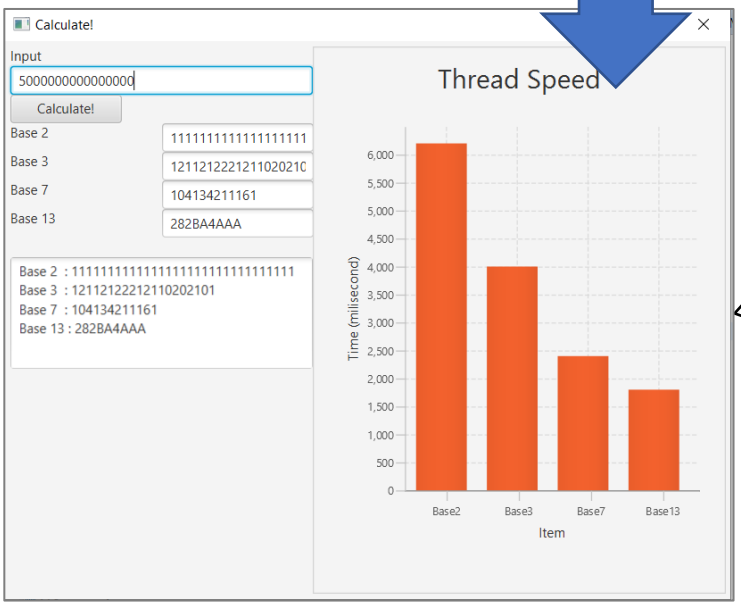


1. Base 13 will be the first to respond. Meanwhile, you can play around with user interface.





2. Then Base 7 will follow, then Base 3. Meanwhile, you can play around with user interface.



3. Base 2 results, text area results, and graph results take a while to show. Meanwhile, you can play around with user interface.

4. Scoring Criteria: (11 points)

4.1. When the Calculate button is pressed (when input is large, such as 5000000000000000).

4.1.1. Calculate button bounces back immediately. **(2 point)**

4.1.2.User can click in all text box and text area while waiting for the results. **(2 points)**

4.1.3. After a short time, result is shown on Base 13 text box first. **(1 points)**

4.1.4.Next, result is shown on Base 7 text box after Base 13. **(1 points)**

4.1.5.Next, result is shown on Base 3 text box after Base 7 and 13. **(1 points)**

4.1.6.After some time, result is shown on Base 2 text box after other base numbers. **(2 points)**

4.1.7.When all results are shown, the text area and graph immediately show their results. **(2 points)**

5. Implementation Detail

The code is given in **main.java**. You do not need to know the details of the user interface, just the components that get updated from the calculation are needed.

- You can add your own methods and variables.
- You **MUST NOT change the code of base_X(double x,int base)**. If you do, you get **0 point**.
- If no threads are used to solve this question, you get **0 point**.

BarChartPane.java class is given, **You MUST NOT change anything in BarChartPane.java.** If you do, you get **0 point**.